Granular RVEs

Three-dimensional computational mechanics of realistic granular microstructures using FEniCSx

Jay M. Appleton

September 12, 2026

Repository	`jay-a/granular-rves`
Source commit	`577bb37`
Generated	2026-09-12T10:29:37.553558+00:00
Author	Jay M. Appleton
Status	Early development
Tests	8 passed / 8 total
Build	passed

Contents

1 Introduction

Granular materials exhibit mechanical behavior that emerges from interactions between individual particles and the evolving network of contacts through which forces are transmitted. Numerical simulation of granular compaction is therefore a challenging computational mechanics problem. During compaction, the contact topology can evolve substantially, frictional contact

can govern the transmission of stresses, and large deformation may lead to highly localized mechanical response and, eventually, particle-scale failure or fracture. These characteristics make the numerical representation of granular assemblies fundamentally different from that of a conventional continuum with a fixed material domain.

Material point methods (MPM) and other particle-based approaches are consequently attractive for many large-deformation granular mechanics problems, since they can accommodate evolving configurations without relying on a conventional mesh that must remain well behaved throughout the deformation. Finite element methods (FEM), however, provide a complementary computational framework in which the continuum formulation, constitutive response, contact treatment, boundary conditions, and numerical solution procedure can be specified and investigated systematically. Rather than assuming a priori that one numerical method is preferable for all granular mechanics problems, this project uses FEM as a controlled computational framework for investigating how far a programmable finite element formulation can be developed for realistic three-dimensional granular assemblies.

The *Granular RVEs* project therefore develops a three-dimensional computational mechanics framework for realistic granular microstructures using FEniCSx. The project begins with well-defined benchmark problems at the individual-grain level and progressively introduces interacting grains, frictional contact, periodic boundary conditions, and realistic three-dimensional grain geometries. This progression is intended to establish a verified computational foundation in which the physical and numerical complexity of the simulations can be increased in a controlled manner.

A central motivation for this development is the eventual comparison of finite element and alternative numerical formulations for granular compaction. Realistic grain geometries and representative volume element (RVE) configurations provide a natural basis for such comparisons because the same microstructure, material parameters, loading conditions, and boundary conditions can in principle be supplied to different numerical frameworks. Once the FEniCSx formulation is sufficiently developed and verified, these matched simulations can support one-to-one comparisons with other approaches, including material point methods, with respect to mechanical response, numerical behavior, computational cost, and practical limitations.

The longer-term objective is to extend the computational framework from direct grain-scale simulation toward homogenization, model parameter identification, and uncertainty quantification. These capabilities would provide a basis for connecting resolved microstructural simulations with reduced-order and data-driven models. In particular, the resulting parameterized finite element simulations may ultimately serve as the high-fidelity model evaluations required for the development and assessment of surrogate models and neural-operator approaches for granular mechanics.

At its current stage, the project focuses on establishing this computational foundation rather than claiming a complete solution to granular compaction. The emphasis is on reproducible scientific software, verification of the underlying numerical formulations, and a progressive increase in physical complexity. This provides a framework in which the capabilities and limitations of the FEniCSx approach can be evaluated rigorously as the project develops.

2 Theory

This section is currently being developed alongside the implementation. The immediate objective is to establish a verified three-dimensional finite element benchmark for compression of a single elastic cylinder between frictionless rigid platens. The benchmark will provide the first working mechanics problem for the project and establish the computational patterns that will later be extended to granular assemblies.

The implementation roadmap is listed below. Each item identifies the principal source files expected to implement that capability. File names may change as the implementation develops;

the list is intended to make the dependency structure of the first simulation explicit.

2.1 Implementation Roadmap

ID	Task	Relevant files	Status
1	Define the 3D cylinder geometry	`problem/geometry/cylinder.py`	TODO
2	Generate or import a tetrahedral mesh	`problem/geometry/cylinder.py`, `io/`	TODO
3	Define small-strain kinematics	`mechanics/kinematics/small_strain.py`	TODO
4	Implement isotropic linear elasticity	`mechanics/constitutive/linear_elastic.py`	TODO
5	Define the compression boundary conditions	`problem/boundary/`	TODO
6	Implement frictionless platen/contact treatment	`mechanics/contact/`, `problem/boundary/`	TODO
7	Define and assemble the variational problem	`mechanics/balance/`, `numerical/`	TODO
8	Solve the linear system with DOLFINx/PETSc	`numerical/`	TODO
9	Extract reaction forces and macroscopic response	`numerical/`, `io/`	TODO
10	Write VTK-compatible simulation output	`io/`	TODO
11	Implement an independent analytical or semi-analytical reference solution	`benchmarks/01_cylinder_compression/reference.py`	TODO
12	Compare the finite element solution with the reference solution	`benchmarks/01_cylinder_compression/`, `tests/`	TODO
13	Perform a mesh-convergence study	`benchmarks/01_cylinder_compression/experiment.yaml`, `tests/`	TODO
14	Add finite-strain kinematics	`mechanics/kinematics/finite_strain.py`	TODO
15	Implement a compressible Neo-Hookean constitutive model	`mechanics/constitutive/neo_hookean.py`	TODO
16	Add the nonlinear finite-strain solver	`numerical/`	TODO
17	Compare small-strain and finite-strain solutions	`benchmarks/01_cylinder_compression/`	TODO
18	Compute the relative L^2 strain-field error	`benchmarks/01_cylinder_compression/`, `io/`	TODO

2.2 Initial Benchmark: 3D Linear Elastic Cylinder

The first benchmark will solve the fully three-dimensional finite element problem for a cylindrical elastic body subjected to displacement-controlled compression between frictionless rigid platens.

The finite element implementation will not use an axisymmetric formulation. Instead, the axisymmetric solution will serve as an independent reference against which the three-dimensional calculation can be verified.

The initial material model will be isotropic, small-strain linear elasticity, parameterized by Young's modulus E and Poisson's ratio ν. The corresponding kinematic, constitutive, and equilibrium relations will be implemented using the mechanics modules identified in the roadmap above.

The benchmark will initially focus on:

- three-dimensional geometry;
- small-strain kinematics;
- isotropic linear elasticity;
- displacement-controlled compression;
- frictionless interaction with the rigid platens;
- reaction-force extraction;
- VTK-compatible output; and
- comparison against an independent analytical or semi-analytical reference.

The purpose of this problem is not to represent granular mechanics yet. It is a controlled computational mechanics problem that allows the basic FEniCSx implementation, boundary treatment, solver configuration, output, and verification workflow to be established before introducing multiple grains, evolving contact networks, periodicity, and realistic grain geometries.

2.3 Reference Solution and Verification

The reference solution will be maintained independently of the core finite element implementation. In particular, the core package will not contain axisymmetric-specific logic merely to support this benchmark.

The reference problem will exploit the axisymmetric structure of the cylindrical compression problem. The resulting displacement and stress fields are generally functions of radial and axial position and are not assumed to be spatially uniform near the platen interfaces. The reference calculation will therefore provide a physically meaningful comparison for the three-dimensional finite element solution rather than reducing the verification problem to a homogeneous uniaxial-stress calculation.

Verification quantities will include the macroscopic reaction force and, where appropriate, displacement and strain-field comparisons. Mesh refinement will be used to determine whether the three-dimensional finite element solution approaches the reference solution systematically.

2.4 Finite-Strain Extension

After the small-strain benchmark has been verified, the same geometric and loading configuration will be extended to finite-strain elasticity. The finite-strain formulation will introduce the deformation gradient and a nonlinear constitutive response, initially using a compressible Neo-Hookean model whose small-strain limit is consistent with the selected linear-elastic material parameters.

The finite-strain calculation will use the nonlinear solution infrastructure provided by DOLFINx and PETSc. The resulting solution will be compared with the small-strain calculation at increasing levels of compression.

The comparison will include:

- reaction force versus applied engineering strain;
- selected stress and strain fields;
- displacement fields; and
- the relative L^2 error between the finite-strain and small-strain strain fields.

The strain-field comparison will be used to identify the deformation range over which the small-strain approximation remains adequate and the range in which finite-strain effects become significant. The interpretation of this comparison will distinguish kinematic effects from constitutive-model differences.

2.5 Planned Benchmark Structure

The first benchmark is expected to be organized independently from the core mechanics package:

```
benchmarks/
‘-- 01_cylinder_compression/     |-- README.md     |-- experiment.yaml
    ‘-- reference.py
```

The benchmark configuration will describe the physical experiment and numerical parameters, while the reusable implementation will remain in the `granular_rves` package. This separation is intended to keep benchmark-specific verification logic out of the general mechanics infrastructure.

As the implementation matures, this section will be replaced progressively by the mathematical formulation, discretization details, solver configuration, verification methodology, and results associated with each completed milestone.

3 Computational Toolchain

3.1 Geometry and Meshing

The geometry layer defines the physical domain of a computational problem and is responsible for producing the finite-element mesh used by the mechanics formulation. Geometry declarations are kept distinct from their numerical realization: the problem definition specifies the physical objects that constitute the model, while the geometry package determines how those objects are constructed, meshed, and converted into the finite-element representation required by DOLFINx.

A computational problem may contain multiple geometry objects. Each object is identified by a semantic name and a geometry type, together with the parameters required to define its physical dimensions and placement. For example, a cylindrical specimen and two rigid platens can be declared independently:

```
geometry:
  - name: specimen
    type: cylinder
    x0: [0.0, 0.0, 0.0]
    x1: [0.0, 0.0, 0.010]
    radius: 0.005

  - name: bottom_platen
    type: cylinder
    x0: [0.0, 0.0, -0.001]
    x1: [0.0, 0.0, 0.0]
    radius: 0.010
```

```
  - name: top_platen
    type: cylinder
    x0: [0.0, 0.0, 0.010]
    x1: [0.0, 0.0, 0.011]
    radius: 0.010
```

The geometry package separates the declaration of these physical objects from mesh generation. The initial implementation is organized around the following responsibilities:

`definitions.py` Defines typed representations of geometry declarations. This layer describes what physical geometry has been requested and does not depend on Gmsh or DOLFINx.

`registry.py` Maps geometry type names, such as `cylinder`, `stl`, or `obj`, to their corresponding geometry implementations.

`builders.py` Provides the common realization interface used to construct geometric models from the typed geometry definitions.

`cylinder.py` Implements analytic cylindrical geometry. A cylinder is defined parametrically by its two end points and radius.

`sources.py` Provides interfaces for geometry originating from external surface or mesh sources, including formats such as STL and OBJ.

`meshing.py` Applies the mesh discretization policy, generates the finite-element mesh, assigns semantic physical groups, and converts the resulting model to the DOLFINx mesh representation.

The resulting package therefore follows the separation

$$\textit{physical geometry declaration} \longrightarrow \textit{geometry realization} \longrightarrow \textit{mesh generation} \longrightarrow \textit{DOLFINx mesh.}$$

The initial discretization strategy uses conforming three-dimensional tetrahedral elements with first-order geometry. This provides a straightforward baseline for irregular three-dimensional grain geometries while leaving higher-order and curved discretizations available for later development. Mesh resolution is specified independently from the physical geometry through global mesh defaults, with optional local overrides for individual objects or features.

Semantic physical groups are retained during mesh generation so that downstream boundary-condition, loading, and contact definitions can refer to physical entities by name rather than by Gmsh-specific integer identifiers. This provides a stable interface between the problem-definition layer and the mechanics formulation.

The geometry package ultimately hands the generated mesh and associated entity tags to DOLFINx. Function-space construction, variational forms, assembly, and solution therefore remain outside the geometry layer. Similarly, adaptive mesh refinement is not treated as a geometry operation; the geometry layer provides the initial discretization, while any solution-driven refinement belongs to the numerical solution workflow.

4 Usage

This section is under development.

5 Computational Experiments

This section is under development.

6 Verification and Validation

This section is under development.

7 Results

This section is under development.

8 Research Direction

This section is under development.

9 Reproducibility

This section is under development.

References